

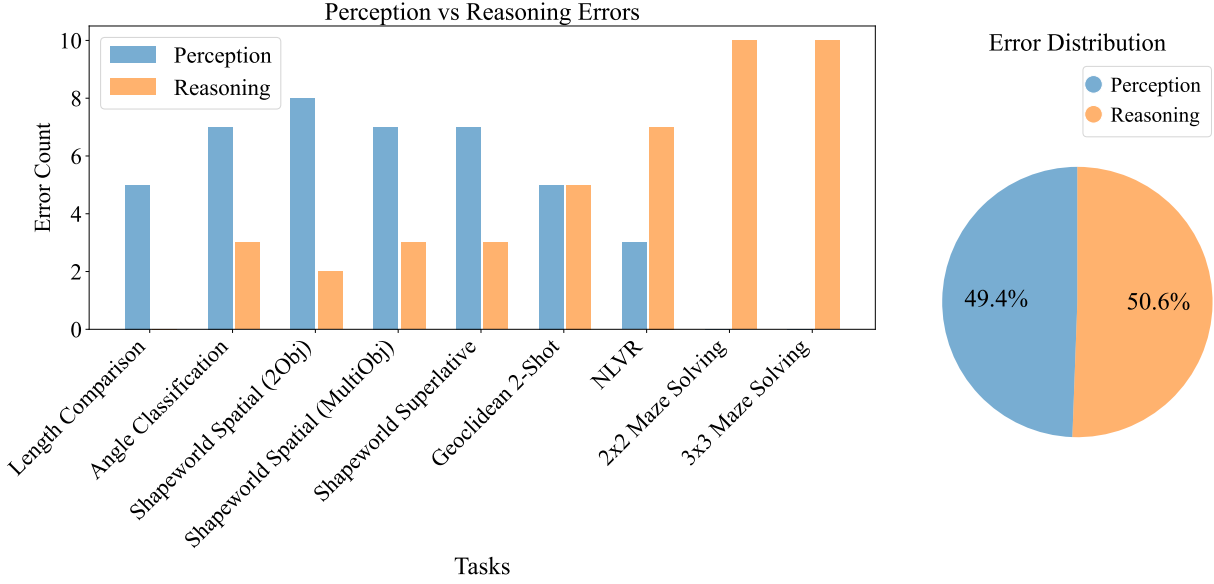


Figure 11: Error distribution by VDLM-txt between perception and reasoning on low-level vector graphics reasoning.

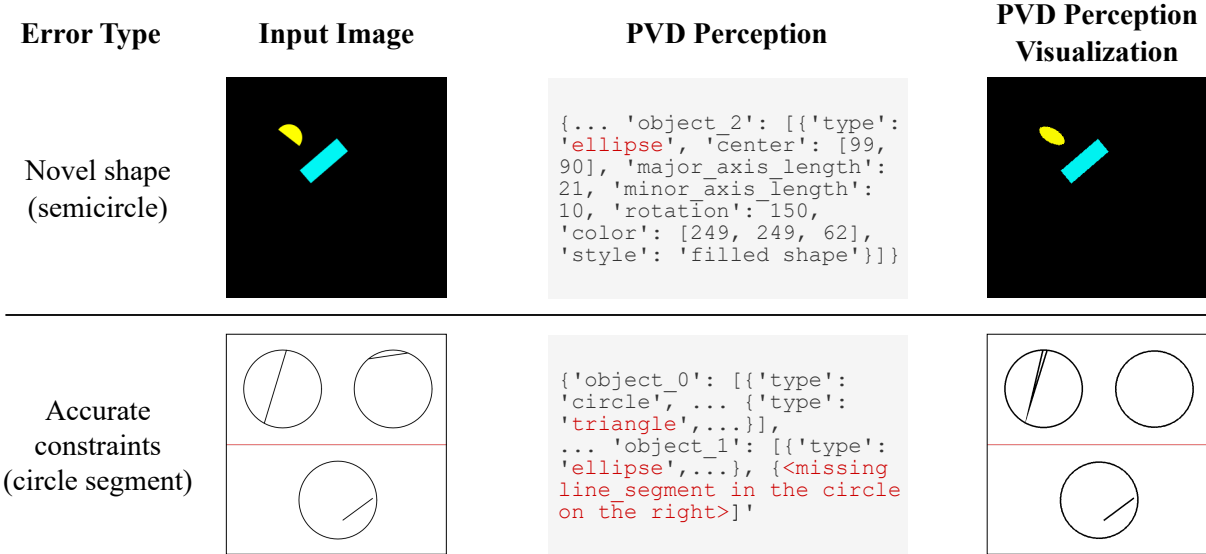


Figure 12: Perception error examples. The example at the top illustrates an error wherein the SVG-to-PVD model predicts a semicircle as an ellipse. The example at the bottom demonstrates that the SVG-to-PVD model struggles to decode overlapping primitives with accurate constraints, such as a segment of a circle.

(2) **Accurate constraints between primitives:** Although the PVD accommodates scenarios where multiple objects of the same color overlap, the attributes, e.g., position, of each object are decided independently and randomly. Thus, the SVG-to-PVD model often fails to capture intentional constraints between objects; for example, a line that perfectly segments a circle. These constraints are particularly emphasized in the Geoclidean 2-shot Learning task (Figure 12), where VDLM struggles to outperform GPT-4V and GPT-4o.

(3) **Very small objects:** During inference, the iterative decomposition process heuristically ignores SVG paths that only contribute only minor differences to the reconstructed image. This method effectively reduces

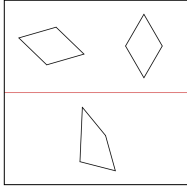
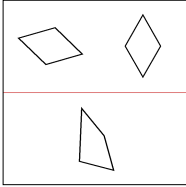
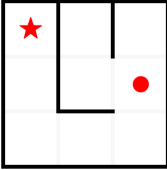
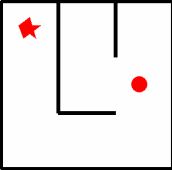
Error Type	Input Image	PVD Perception Visualization	Reasoning Model (GPT-4) Response
Discovering constraints (Rhombus v.s. Quadrilateral)			To analyze the given JSON and identify the pattern and constraints of the reference and test concepts, let's break down the information provided for each object: Object 0** (Test Example): - Type: Polygon - Vertices: [(304, 416), (395, 527), (433, 665), (295, 629)] - Characteristics: Outlined shape, black color, polygon with 4 vertices. Object 1** (Reference Example 1): - Type: Polygon ... Object 2** (Reference Example 2): - Type: Polygon ... Given the descriptions, the reference examples (Object 1 and Object 2) and the test example (Object 0) are all polygons with 4 vertices, suggesting they are quadrilaterals... Given the information and assuming the concept involves quadrilaterals without requiring them to have uniform angles or sides (since we cannot precisely measure them from the given data), the test example appears to follow the same broad concept of being a quadrilateral as the reference examples.
Algorithmic reasoning (Inferring maze connectivity)			By examining the grid lines, we can determine where the walls are and thus infer the connections. However, the JSON doesn't directly specify the walls between cells, but we can deduce this from the absence of grid lines between cells. For simplicity, let's assume a fully connected grid without internal walls (as the JSON doesn't specify walls between cells explicitly), and then adjust based on typical maze patterns: Vertical Connections:** All true, indicating no vertical walls between cells. Horizontal Connections:** All true, indicating no horizontal walls between cells. ... ["solution": "[(1, 1), (0, 0)]"]

Figure 13: Reasoning error examples: The example at the top demonstrates that the reasoning model fails to uncover the deeper constraints within the perceived polygons. It is able to recognize that they are all quadrilaterals but unable to further discern that the reference concepts are rhombuses with four equal sides, while the test concept is not. The example at the bottom illustrates that the reasoning model struggles to infer connectivity based on the perceived grid, thus failing to provide the correct solution.

noise from the rule-based image-to-SVG converter but may omit very small objects in some cases. Adjusting this threshold is necessary for specific scenarios.

Common reasoning errors. (1) **Discovering intentional constraints:** Without specific queries, the reasoning model can fail to identify intentional constraints. For example, differentiating a rhombus from a general quadrilateral, as shown in Figure 13.

(2) **Handling ambiguity:** Visual inputs sometimes provide useful inductive biases that can help the model better understand the task or make reasonable assumptions when the instructions are ambiguous. For instance, when presenting an angle in an image and asking whether it is an acute or obtuse angle, as in Figure 4, it is visually straightforward to assume that the angle is defined by the middle point as the vertex with rays extending outwards. However, without such visual cues, reasoning over pure symbolic representations makes it challenging to infer which angle the question refers to among the detected undirected edges. To mitigate ambiguity, adding more precise instructions for VDLM-txt is necessary in some tasks.

(3) **Algorithmic reasoning:** Language-based reasoners can struggle with complex multi-step reasoning tasks, such as inferring the connectivity (Figure 13) of a maze using the vertices and edges of the grid in pixel coordinates, or counting the number of objects located within a certain box.

C Primal Visual Description (PVD) Details

PVD JSON schema definition: See Figure 14.

Types	Schema	Example
Circle	<pre>{ "type": "circle", "center": [x, y], "radius": r, "color": [r, g, b], "style": "filled shape" or "outlined shape", "line_width": d (if style is "outlined") }</pre>	<pre>{ "type": "circle", "center": [205, 210], "radius": 117, "color": [193, 190, 165], "style": "outlined shape", "line_width": 9 }</pre>
Ellipse	<pre>{ "type": "ellipse", "center": [x, y], "major_axis_length": l1, "minor_axis_length": l2, "rotation": o, "color": [r, g, b], "style": "filled shape" or "outlined shape", "line_width": d (if style is "outlined") }</pre>	<pre>{ "type": "ellipse", "center": [278, 166], "major_axis_length": 147, "minor_axis_length": 60, "rotation": 16, "color": [85, 220, 98], "style": "filled shape" }</pre>
Rectangle	<pre>{ "type": "rectangle" or "triangle" or "polygon", "vertices": [[x1, y1], [x2, y2], ...], "color": [r, g, b], "style": "filled shape" or "outlined shape", "line_width": d (if style is "outlined") }</pre>	<pre>{ "type": "triangle", "vertices": [[452, 418], [298, 113], [266, 255]], "color": [165, 170, 141], "style": "filled shape", }</pre>
Triangle		
Polygon		
Line Segment	<pre>{ "type": "line_segment", "vertices": [[x1, y1], [x2, y2]] "color": [r, g, b], "line_width": d }</pre>	<pre>{ "type": "line_segment", "vertices": [[822, 114], [93, 20]], "color": [166, 32, 97], "line_width": 10 }</pre>
Grid	<pre>{ "type": "grid", "vertices": [[x1, y1], [x2, y2], ...], "edges": [[x1, y1], [x2, y2]], ...], "color": [r, g, b], "line_width": d }</pre>	<pre>{ "type": "grid", "vertices": [[73, 214], [73, 640], [215, 214], [215, 640]], "edges": [[[73, 214], [73, 640]], [[215, 214], [215, 640]], [[73, 640], [215, 640]]], "color": [23, 31, 120], "line_width": 3 }</pre>
Path	<pre>{ "type": "path", "vertices": [[x1, y1], [x2, y2], ...], "edges": [[x1, y1], [x2, y2]], ...], "color": [r, g, b], "line_width": d }</pre>	<pre>{ "type": "path", "vertices": [[59, 69], [17, 330], [61, 77]], "edges": [[[59, 69], [17, 330]], [[17, 330], [61, 77]]], "color": [98, 28, 0], "line_width": 5 }</pre>
Graph	<pre>{ "type": "line drawing", "vertices": [[x1, y1], [x2, y2], ...], "edges": [[x1, y1], [x2, y2]], ...], "color": [r, g, b], "line_width": d }</pre>	<pre>{ "type": "line drawing", "vertices": [[399, 497], [433, 823], [483, 570], [531, 443], [534, 578]], "edges": [[[399, 497], [483, 570]], [[531, 443], [534, 578]], [[483, 570], [534, 578]], [[483, 570], [433, 823]], [[534, 578], [433, 823]]], "color": [254, 230, 139], "line_width": 9 }</pre>

Figure 14: PVD JSON schema definition.

Generation procedures (Single Object):

- **Circle:** Randomly sample a center and a radius to draw a circle within the canvas.